



MILESTONE 5

Nightly Automation

Orchestrating the full Bronze to Gold pipeline as one scheduled, idempotent nightly job



STEP 1

Secure the Credentials

WHAT

Stored the Databricks access token in Azure Key Vault and created a Databricks secret scope backed by that vault. The orchestration notebook reads the token via `dbutils.secrets.get`, never hard-coded.

WHY

Secrets never belong in code or in Git. A Key Vault-backed scope keeps the token in one governed store, read at runtime by the AzureDatabricks identity (granted the Key Vault Secrets User role). Least-privilege token scoped to SQL, Clusters, and Jobs.

1

STEP 1 · SCREENSHOT

Secure the Credentials

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P dbw-atliq-ab

+ New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

Create Secret Scope

A store for secrets that is identified by a name and backed by a specific store type. [Learn more](#)

Scope Name ⁱ

atliq

Manage Principal ⁱ

All workspace users

Azure Key Vault ⁱ

DNS Name

https://kv-atliq-ab.vault.azure.net/

Resource ID

/subscriptions/fe7a04e5-37b1-4992-a127-541400ea45ad/resourceGroups/rg-atliq-capstone

Cancel Create

The secret scope named atliq has been added. Manage secrets in this scope in Azure KeyVault with manage principal = users

OK

Create Secret Scope: atliq scope, backed by kv-atliq-ab, added



STEP 2

Wrap dbt in a Notebook

WHAT

Built a `run_dbt` notebook that installs dbt on the cluster, writes a `runtime profiles.yml` from the secret token, runs dbt deps, then dbt build against the Gold project in the Git folder.

WHY

ADF orchestrates by calling Databricks notebooks, so dbt has to run as one. Reading the dbt project from the connected Git folder keeps a single source of truth, and the `runtime profiles.yml` is never committed.

Wrap dbt in a Notebook

The screenshot shows a Databricks notebook interface. The top bar indicates the environment is Microsoft Azure and the workspace is databricks. The notebook title is 'run_dbt'. The code is written in Python and uses the subprocess module to run dbt commands. The output shows the successful execution of dbt build, with 26 tests passing and no errors.

```
import subprocess

# install packages first (dbt_utils, needed for the accepted_range test)
deps = subprocess.run(["dbt", "deps", "--project-dir", dbt_project_dir], capture_output=True, text=True)
print(deps.stdout, deps.stderr)

# then build models + run tests
result = subprocess.run(
    ["dbt", "build", "--project-dir", dbt_project_dir, "--profiles-dir", dbt_project_dir],
    capture_output=True, text=True
)
print(result.stdout)
print(result.stderr)
if result.returncode != 0:
    raise Exception("dbt build failed")
print("dbt build succeeded")
```

Output logs:

```
18:29:02 25 of 26 START test relationships_fact_sales_product_id__product_id__ref_dim_product_ [RUN]
18:29:02 23 of 26 PASS not_null_dim_product_product_id ..... [PASS in 0.89s]
18:29:02 26 of 26 START test unique_dim_product_product_id ..... [RUN]
18:29:02 21 of 26 PASS relationships_fact_sales_date_key__date_key__ref_dim_date_ ..... [PASS in 1.27s]
18:29:02 24 of 26 PASS not_null_dim_product_unit_price ..... [PASS in 0.85s]
18:29:03 26 of 26 PASS unique_dim_product_product_id ..... [PASS in 0.85s]
18:29:03 25 of 26 PASS relationships_fact_sales_product_id__product_id__ref_dim_product_ [PASS in 1.18s]
18:29:03
18:29:03 Finished running 4 table models, 16 data tests, 6 view models in 0 hours 0 minutes and 28.07 seconds (28.07s).
18:29:03
18:29:03 Completed successfully
18:29:03
18:29:03 Done. PASS=26 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=26
18:29:03 [WARNING][DeprecationsSummary]: Deprecated functionality
Summary of encountered deprecations:
- MissingArgumentsPropertyInGenericTestDeprecation: 5 occurrences
To see all deprecation instances instead of just the first occurrence of each,
run command again with the '--show-all-deprecations' flag. You may also need to
run with '--no-partial-parse' as some deprecations are only encountered during
parsing.

dbt build succeeded
```

run_dbt notebook: dbt build Completed successfully, PASS=26 ERROR=0



STEP 3

Connect ADF to Databricks

WHAT

Created a Databricks linked service in ADF pointing at the workspace, using a job cluster and reading the token from Key Vault. The token is scoped to SQL, Clusters, and Jobs so ADF can create clusters and run notebooks.

WHY

This is the bridge that lets the ADF pipeline call the Databricks notebooks. A job cluster spins up per run and terminates after, so no compute is left running between nightly jobs.

Connect ADF to Databricks

The screenshot displays the Microsoft Azure portal interface for configuring a linked service. The left sidebar shows the navigation menu with 'Connections' > 'Linked services' selected. The main pane shows a list of linked services: 'ls_adls' (Azure Data Lake Storage Gen2), 'ls_azuresql' (Azure SQL Database), 'ls_databricks' (Azure Databricks), and 'ls_keyvault' (Azure Key Vault). The 'ls_databricks' service is selected, and the 'Edit linked service' pane on the right is open.

Edit linked service
Azure Databricks [Learn more](#)

Name *
ls_databricks

Description

Connect via integration runtime *
☒ AutoResolveIntegrationRuntime

Account selection method *
☐ From Azure subscription ☒ Enter manually

Databrick Workspace URL *

Authentication type *

Access token **Azure Key Vault**

AKV linked service *

Secret name *

☒ Edit

Secret version

☐ Edit

Select cluster

Cluster version *

Cluster node type *

Python version *

Worker options
☒ Fixed ☐ Autoscaling

Save **Cancel**

Connection successful
[Test connection](#)

ls_databricks: token from Key Vault (dbt-token), new job cluster, Test connection successful



STEP 4

Chain the Full Pipeline

WHAT

Extended the pipeline (renamed `pl_nightly_sync`) so that after Bronze ingestion it runs three Databricks notebooks in order: Silver full dims, Silver incremental (MERGE), and `run_dbt`. Silver waits for all Bronze branches to succeed.

WHY

One pipeline orchestrates the whole nightly chain: ingest to Bronze, transform to Silver, build Gold. Success dependencies mean a failed Bronze step stops the run instead of feeding Silver incomplete data.

Chain the Full Pipeline

The screenshot displays the Microsoft Azure Data Factory (ADF) interface for a pipeline named `pl_nightly_sync`. The left sidebar shows the 'Factory Resources' tree with categories like Pipelines, Datasets, Data flows, Power Query, and Templates. The 'Activities' pane on the right lists available activities such as Notebook, Synapse, and Databricks. The main canvas shows the pipeline logic, which starts with a 'Set variable' activity (Set run_start_at), followed by a 'Lookup' activity (Lookup control table). This is followed by two parallel 'Copy data' activities (Copy supplier CSV and Copy marketing CSV), which then merge into a 'ForEach' activity (ForEach source). The 'ForEach' activity leads to a 'Notebook' activity (Silver full dims), which then flows into another 'Notebook' activity (Silver incremental), and finally into a third 'Notebook' activity (Run dbt). The bottom pane shows the 'Parameters' tab with a '+ New' button.

pl_nightly_sync: ingestion, then Silver full dims, Silver incremental, Run dbt

Chain the Full Pipeline

The screenshot displays the Microsoft Azure Data Factory console. The top navigation bar shows the user 'ashish@alphainterbiz.com' and the data factory 'adf-atliq-ab'. The left sidebar lists 'Factory Resources' including Pipelines, Datasets, Data flows, Power Query, and Templates. The 'Pipelines' section is expanded, showing the pipeline 'pl_nightly_sync'. The main canvas displays the pipeline's logic flow, which includes a 'Set variable' activity, a 'Lookup' activity, a 'Copy data' activity, a 'ForEach' loop, and several 'Notebook' activities. The pipeline is currently in a 'Validated' state. The bottom panel shows the 'Settings' tab for the pipeline, with the 'Notebook path' set to '/Workspace/Users/ashish@alphainterbiz.com...'. The right sidebar displays the 'Pipeline validation output', which states: 'Your pipeline has been validated. No errors were found.' A 'Close' button is visible at the bottom right of the validation output panel.

pl_nightly_sync validated: no errors found



STEP 5

Run End-to-End

WHAT

Ran the whole pipeline in one execution: Set variable, Lookup, ForEach ingestion, CSV copies, Silver full dims, Silver incremental, and Run dbt, all succeeding.

WHY

A single green run proves the entire lakehouse pipeline works as one orchestrated unit, not just as separate milestones. This is the nightly job that would run unattended in production.

Run End-to-End

Microsoft Azure | Data Factory | adf-atliq-ab

Search

ashish@alphainterbiz.com ALPHA INTERBIZ LLP

Upgrade to Fabric (preview)

main branch Validate all Save all Publish

Factory Resources

Filter resources by name

Pipelines 1

p_l_nightly_sync

Change Data Capture (preview) 0

Datasets 5

Data flows 0

Power Query 0

Templates 0

Activities

Search activities

Move and transform

Synapse

Azure Data Explorer

Azure Function

Batch Service

Databricks

General

HDInsight

Iteration & conditionals

Machine Learning

Power Query

Parameters Variables Settings Output

Showing 1 - 21 of 21 items

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime	User prop...	Activity run ID
Run dbt	✓ Succeeded	Notebook	9/20/2026, 3:34:24 AM	4m 10s	AutoResolveIntegrationRuntime (Central India)		227d247c-732f-4c6c-96c4-e6
Silver incremental	✓ Succeeded	Notebook	9/20/2026, 3:30:18 AM	4m 5s	AutoResolveIntegrationRuntime (Central India)		e3e52971-bd40-4474-8b39-c
Silver full dims	✓ Succeeded	Notebook	9/20/2026, 3:23:06 AM	7m 11s	AutoResolveIntegrationRuntime (Central India)		ecd082fc-32ce-4b5e-bc3f-b2
Copy full	✓ Succeeded	Copy data	9/20/2026, 3:22:41 AM	19s	AutoResolveIntegrationRuntime (Central India)		113edc26-c977-4f8a-ba21-2t
If incremental	✓ Succeeded	If Condition	9/20/2026, 3:22:40 AM	23s			5d67d7a8-e444-44be-b9cb-5
Advance watermark	✓ Succeeded	Stored procedu	9/20/2026, 3:22:33 AM	12s	AutoResolveIntegrationRuntime (Central India)		1c0a285c-2dc1-4333-ba89-0
Advance watermark	✓ Succeeded	Stored procedu	9/20/2026, 3:22:32 AM	14s	AutoResolveIntegrationRuntime (Central India)		c7846846-ca41-457a-ade5-d
Advance watermark	✓ Succeeded	Stored procedu	9/20/2026, 3:22:32 AM	14s	AutoResolveIntegrationRuntime (Central India)		5f2a87f6-ec30-4ad3-ba6e-25
Copy full	✓ Succeeded	Copy data	9/20/2026, 3:22:14 AM	24s	AutoResolveIntegrationRuntime (Central India)		3657930e-b68d-463a-9bd9-2
Copy incremental	✓ Succeeded	Copy data	9/20/2026, 3:22:12 AM	18s	AutoResolveIntegrationRuntime (Central India)		0e47a165-557d-4cc4-90ca-ac
Copy incremental	✓ Succeeded	Copy data	9/20/2026, 3:22:12 AM	20s	AutoResolveIntegrationRuntime (Central India)		bc8b6e0e-2517-4a77-830a-b
Copy incremental	✓ Succeeded	Copy data	9/20/2026, 3:22:12 AM	19s	AutoResolveIntegrationRuntime (Central India)		80427e4e-1642-43dd-8293-7
If incremental	✓ Succeeded	If Condition	9/20/2026, 3:22:12 AM	36s			614ddc9f-0dfa-41cc-8063-42
If incremental	✓ Succeeded	If Condition	9/20/2026, 3:22:12 AM	27s			4569ba2b-e744-4d22-a3bd-4
If incremental	✓ Succeeded	If Condition	9/20/2026, 3:22:12 AM	36s			98058543-1663-4194-9809-2
If incremental	✓ Succeeded	If Condition	9/20/2026, 3:22:12 AM	36s			1034a65a-f19d-4c6d-90a1-5
ForEach source	✓ Succeeded	ForEach	9/20/2026, 3:22:11 AM	54s			2c11e298-5619-4163-b146-a
Copy marketing CSV	✓ Succeeded	Copy data	9/20/2026, 3:21:53 AM	19s	AutoResolveIntegrationRuntime (Central India)		503c834c-13fe-4a21-a399-79
Copy supplier CSV	✓ Succeeded	Copy data	9/20/2026, 3:21:53 AM	19s	AutoResolveIntegrationRuntime (Central India)		6cf77224-1f91-4342-bc2b-38
Lookup control_table	✓ Succeeded	Lookup	9/20/2026, 3:21:53 AM	17s	AutoResolveIntegrationRuntime (Central India)		c1a9e310-a31b-4857-98a9-fc
Set run_start_at	✓ Succeeded	Set variable	9/20/2026, 3:21:52 AM	Less than 1s			8c21fe38-38f1-49ad-8726-29

Pipeline run output: every activity Succeeded, Bronze through Run dbt



STEP 6

Schedule the Nightly Run

WHAT

Added a schedule trigger (daily at 01:00) to the pipeline, then paused it after capturing the configuration so it does not run unattended on paid compute during the capstone.

WHY

The schedule is what makes the sync a nightly job rather than a manual one. Pausing after capture demonstrates the setup while keeping cost controlled, the real trade-off when a trigger runs a paid job cluster.

6

STEP 6 • SCREENSHOT

Schedule the Nightly Run

The screenshot shows the Microsoft Azure Data Factory 'Edit trigger' interface for a trigger named 'trg_nightly'. The interface is divided into three main sections: a left-hand navigation pane, a central 'Triggers' list, and a right-hand 'Edit trigger' form.

Left-hand navigation pane: Contains various settings and tools, including 'General', 'Connections', 'Source control', 'Author', 'Global parameters', 'Data flow libraries', 'Security', 'Workflow orchestration manager', and 'Apache Airflow'.

Central 'Triggers' section: Displays a table of triggers. The table has columns for 'Name', 'Type', and 'Status'. The 'trg_nightly' trigger is listed with a 'Schedule' type and a 'Stopped' status.

Right-hand 'Edit trigger' form: Contains the following fields and options:

- Name:** A text input field containing 'trg_nightly'.
- Description:** A text area for describing the trigger.
- Type:** A dropdown menu set to 'ScheduleTrigger'.
- Start date:** A date and time picker set to '9/19/2026, 9:02:00 PM'.
- Time zone:** A dropdown menu set to 'Chennai, Kolkata, Mumbai, New Delhi (UTC+5:30)'.
- Recurrence:** A dropdown menu set to 'Every 1 Day(s)'.
- Advanced recurrence options:** A section with 'Execute at these times' and input fields for 'Hours' (set to 1) and 'Minutes' (set to 0).
- Schedule execution times:** A text input field set to '01:00'.
- Specify an end date:** A checkbox that is currently unchecked.
- Annotations:** A section with a '+ New' button.
- Status:** Radio buttons for 'Started' and 'Stopped', with 'Stopped' selected.

At the bottom of the 'Edit trigger' form are 'OK' and 'Cancel' buttons.

trg_nightly: Schedule trigger, every 1 day at 01:00 (UTC+5:30), status Stopped



STEP 7

Prove Idempotency

WHAT

Ran the full pipeline twice with no new source data. Both runs left fact_sales at 783 rows and total gross_revenue at 2,076,041.00, identical.

WHY

This is the core reliability property, graded directly. Watermark ingestion pulls only new rows, Silver MERGE upserts on the business key, and dbt rebuilds Gold from Silver, so re-running the whole chain never double-counts. Safe to run every night, safe to retry.

7

STEP 7 · SCREENSHOT 1 of 2

Prove Idempotency

The screenshot shows the Databricks workspace interface. At the top, there's a search bar and a navigation bar with tabs for 'Home', '00_uc_setup', '20_silver_incremental_merge', 'run_dbt', and 'New Query 2026-09-20 03:05:42'. The 'New Query' tab is active, showing a SQL query editor with the following query:

```
1 | SELECT COUNT(*) AS rows, SUM(gross_revenue) AS total_revenue FROM atliq.gold.fact_sales;
```

Below the query editor, there's a 'Run selected (1000)' button and a status bar indicating '3 minutes ago (8s)' and 'dbw_atliq_ab default'. The results table is displayed below the query, showing 1 row with 783 rows and a total revenue of 2076041.00.

	rows	total_revenue
1	783	2076041.00

At the bottom of the interface, the status bar shows '1 row', '8.083s runtime', and a link to 'See performance (1)'. It also indicates 'Checked for improvements' and 'Refreshed 3m ago'.

Run 1 (baseline): fact_sales 783 rows, total revenue 2,076,041.00

7

STEP 7 • SCREENSHOT 2 of 2

Prove Idempotency

The screenshot displays the Databricks SQL Editor interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and a user profile icon. The left sidebar contains a navigation menu with categories like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main editor area shows a SQL query: `SELECT COUNT(*) AS rows, SUM(gross_revenue) AS total_revenue FROM atliq.gold.fact_sales;`. Below the query, a table view displays the results of the query execution. The table has two columns: `rows` and `total_revenue`. The first row shows 783 rows and a total revenue of 2076041.00. The bottom status bar indicates that the query was executed successfully, showing 1 row, a runtime of 0.375s, and a link to see performance details.

	rows	total_revenue
1	783	2076041.00

Run 2: fact_sales still 783 rows, total revenue 2,076,041.00

Milestone 5 Complete

What this milestone delivered

One orchestrated pipeline

A single ADF pipeline runs the full chain: Bronze ingest, Silver transform, Gold dbt build, calling Databricks notebooks in order.

Scheduled and secure

Daily schedule trigger, token secured in Key Vault via a secret scope, least-privilege scoping, no credentials in code or Git.

Proven idempotent

Two full runs left fact_sales identical (783 rows, 2,076,041.00 revenue). The nightly sync never double-counts.

Idempotency proof

783

fact rows (run 1)

783

fact rows (run 2)

2,076,041

revenue, both runs

0

duplicated rows

Engineering decisions: least-privilege token (SQL, Clusters, Jobs) over All APIs; token in Key Vault, never in code; job cluster that terminates per run; trigger paused after capture to control cost. Serverless SQL auto-pause required waking the database before a run, a real operational note for orchestrating auto-pausing compute.